

Design and Implement a Dimensional Model

Certification Track: DP-600 [Fabric Analytics Engineer Associate]

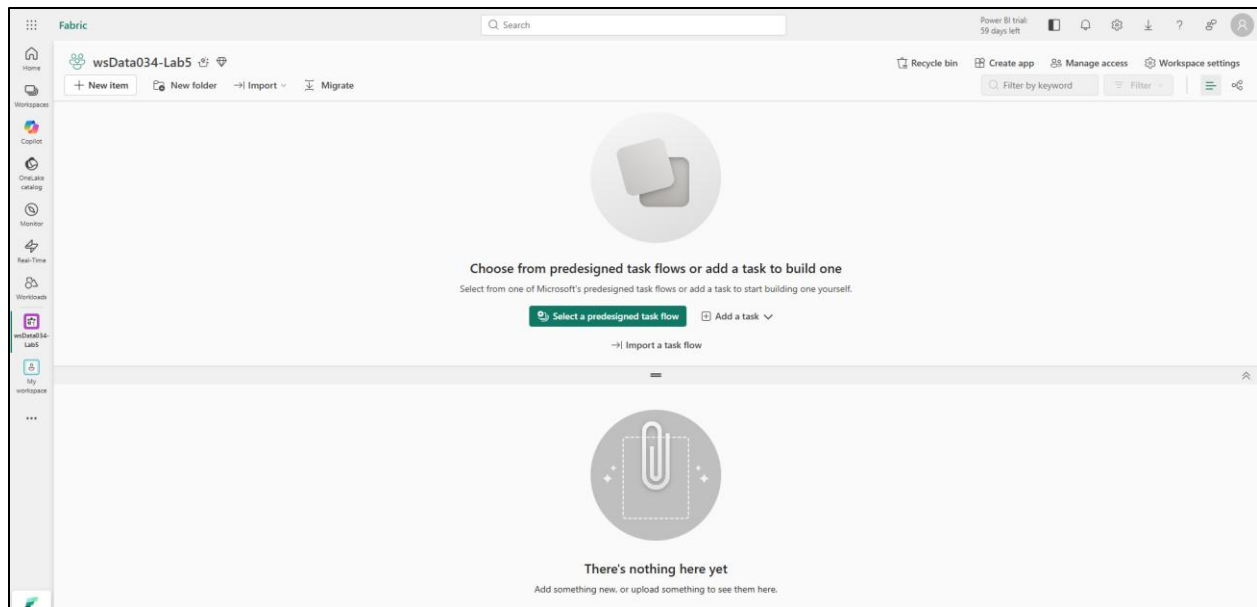
Microsoft Fabric is an analytics platform that structures data using dimensional modeling for extremely fast read performance in heavy analytical workloads. Dimensional models are designed to organize data into two types of tables: a fact table that stores numeric measures and dimension tables that store descriptive attributes about the data in the fact table. The tables are arranged in a star schema that delivers fast queries, simpler relationships and usability. With business users in mind, the star schema structure aligns data to business questions enabling clearer insights and faster decision-making.

In this lab, you build a star schema dimensional model for Contoso Retail to analyze sales across stores, products, customers, and time. By the end of this exercise, you will understand why dimensional modeling, specifically the star schema, is the standard approach for BI workloads and reporting. It delivers fast performance, consistent definitions, and an analytical model that scales with the business.

1. Create Workspace

This is the starting point for any data project in Fabric. The first step is to create a workspace to hold and organize all your Fabric items.

- Open Microsoft Fabric in your web browser and go to the Fabric home page:
<https://app.fabric.microsoft.com/home?experience=fabric>
- Select Workspace from the left menu.
- Click +New workspace.
- Name your workspace and select Create. Your new workspace should be empty.

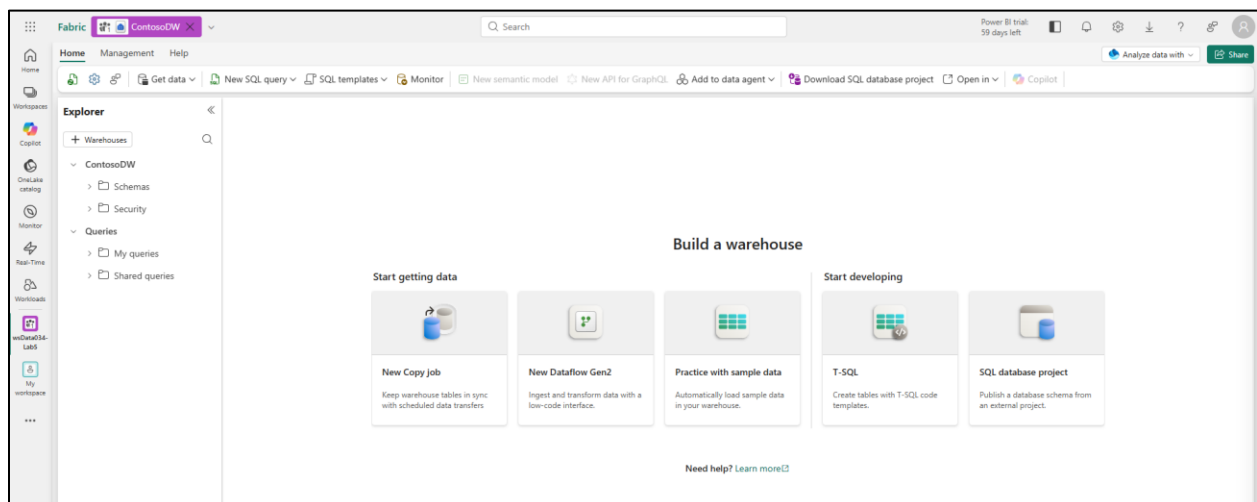


2. Create Data Warehouse

In Fabric, a data warehouse is the traditional home for star schemas because data warehouses are built specifically for analytical workloads, but dimensional models can be built on any platform that supports structured tables.

Once your new workspace has been setup, the second step is to create a data warehouse to store your dimensional model.

- From the top left, select +New Item.
- Scroll down to the Data Warehouse section and select Warehouse.
- Name your warehouse ContosoDW and select Create.



3. Create the Fact Table

A star schema is the most common dimensional model design where a single fact table is placed at the centre and is linked directly to multiple surrounding dimension tables, forming a star-shaped layout that is simple and easy to navigate and understand. The fact table stores numeric measures for the business, does not use a primary key, and has a tall and narrow structure because it grows continuously and contains a very large number of rows.

Understanding the Fact Table:

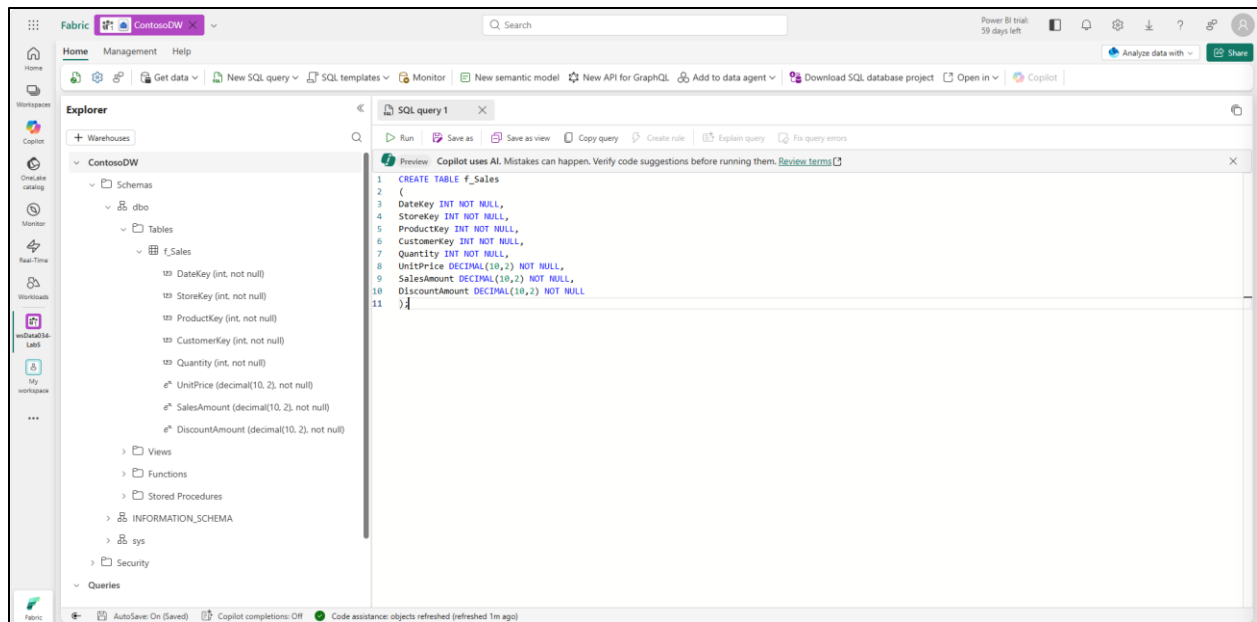
- Measures are numeric columns you aggregate in queries (Quantity, UnitPrice, SalesAmount, DiscountAmount).
- It saves sales transaction line items.
- Each row equals to one line item.
- Foreign keys link each transaction to four dimension tables (Date, Store, Product, Customer).
- The f_prefix marks it as a fact table (f_Sales).

Steps to create the Fact Table:

- In your warehouse, select New SQL query on the toolbar.
- Enter the T-SQL code below and press ► Run button to run the script.

```
CREATE TABLE f_Sales
(
    DateKey INT NOT NULL,
    StoreKey INT NOT NULL,
    ProductKey INT NOT NULL,
    CustomerKey INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    SalesAmount DECIMAL(10,2) NOT NULL,
    DiscountAmount DECIMAL(10,2) NOT NULL
);
```

- Select Refresh on toolbar.
- From the Explorer pane, expand Schemas > dbo > Tables to confirm that the f_Sales table has been created.



4. Create the Dimension Tables

Dimension tables store descriptive information about the business and add meaning to the fact table by answering real business questions such as who, what, when, and where. A dimension table is intentionally denormalized, flattened into a short and wide structure and contains fewer rows because it changes slowly and is only updated occasionally. It lets business users slice and filter data directly from one wide table with many columns instead of using multiple normalized tables in a transactional system.

Understanding the Dimension Tables:

- Dimension tables add context by explaining the details behind each fact (who, what, when, where).
- For this model, you need four dimensions: Date, Store, Product, Customer
- There are two types of Slowly Changing Dimensions (SCD) patterns: Type 1 (overwrite) and Type 2 (historical tracking)
- The Product and Store dimensions track history (SCD Type 2). They include columns ValidFrom, ValidTo, and IsCurrent which keep past versions of products and stores.
- The Customer and Date dimensions use simpler structures. Customer only needs corrections (SCD Type 1). Date is static reference data, so SCD tracking is not needed.
- DateKey uses YYYYMMDD format. This format is efficient and meaningful for date dimensions.

Steps to create the Dimension Tables:

- Still in your warehouse, select New SQL query on the toolbar.
- Enter the T-SQL code below and press ► Run button to run the script.

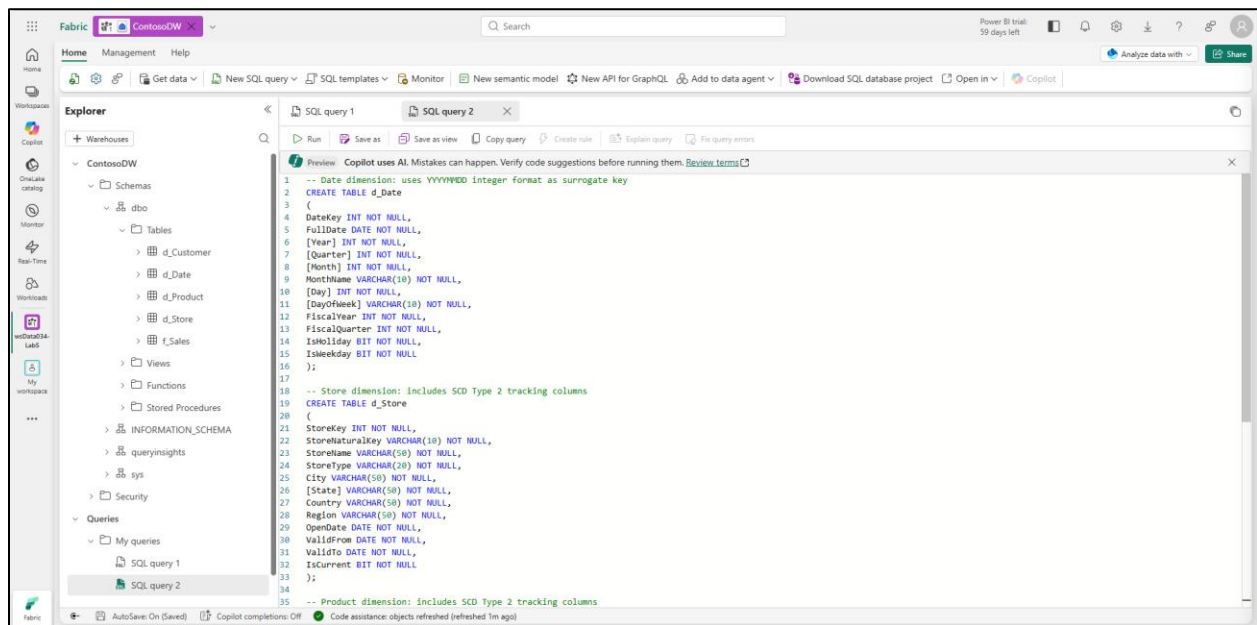
```
-- Date dimension: Uses YYYYMMDD integer format as surrogate key
CREATE TABLE d_Date
(
    DateKey INT NOT NULL,
    FullDate DATE NOT NULL,
    [Year] INT NOT NULL,
    [Quarter] INT NOT NULL,
    [Month] INT NOT NULL,
    MonthName VARCHAR(10) NOT NULL,
    [Day] INT NOT NULL,
    [DayOfWeek] VARCHAR(10) NOT NULL,
    FiscalYear INT NOT NULL,
    FiscalQuarter INT NOT NULL,
    IsHoliday BIT NOT NULL,
    IsWeekday BIT NOT NULL
);

-- Store dimension: Includes SCD Type 2 tracking columns
CREATE TABLE d_Store
(
    StoreKey INT NOT NULL,
    StoreNaturalKey VARCHAR(10) NOT NULL,
    StoreName VARCHAR(50) NOT NULL,
    StoreType VARCHAR(20) NOT NULL,
    City VARCHAR(50) NOT NULL,
    [State] VARCHAR(50) NOT NULL,
    Country VARCHAR(50) NOT NULL,
    Region VARCHAR(50) NOT NULL,
    OpenDate DATE NOT NULL,
    ValidFrom DATE NOT NULL,
    ValidTo DATE NOT NULL,
    IsCurrent BIT NOT NULL
);

-- Product dimension: Includes SCD Type 2 tracking columns
CREATE TABLE d_Product
(
    ProductKey INT NOT NULL,
    ProductNaturalKey VARCHAR(10) NOT NULL,
    ProductName VARCHAR(50) NOT NULL,
    Brand VARCHAR(50) NOT NULL,
    Subcategory VARCHAR(50) NOT NULL,
    Category VARCHAR(50) NOT NULL,
    UnitCost DECIMAL(10,2) NOT NULL,
    ValidFrom DATE NOT NULL,
    ValidTo DATE NOT NULL,
    IsCurrent BIT NOT NULL
);
```

```
-- Customer dimension: Simple structure (SCD Type 1 only)
CREATE TABLE d_Customer
(
    CustomerKey INT NOT NULL,
    CustomerName VARCHAR(50) NOT NULL,
    Segment VARCHAR(20) NOT NULL,
    City VARCHAR(50) NOT NULL,
    [State] VARCHAR(50) NOT NULL,
    Country VARCHAR(50) NOT NULL,
    LoyaltyTier VARCHAR(20) NOT NULL,
    JoinDate DATE NOT NULL
);
```

- Select Refresh on toolbar.
- From the Explorer pane, expand Schemas > dbo > Tables to confirm that all five tables appear (f_Sales, d_Date, d_Store, d_Product, d_Customer).



5. Add Table Constraints

After creating your fact and dimension tables, you can now connect them into a star schema by adding primary keys to each dimension table and foreign keys to the fact table. First use CREATE TABLE (done in step 4), then use ALTER TABLE to add table constraints (primary keys and foreign keys).

Understanding Table Constraints:

- Add primary keys- use ALTER TABLE to assign a primary key to each dimension table (Date, Store, Product, Customer). These keys uniquely identify each dimension row.
- Add foreign keys- use ALTER TABLE to assign a foreign key to the fact table (f_Sales) so each row links back to the correct dimension table. These constraints are metadata only, they are NOT ENFORCED.
- This metadata makes your warehouse tables understandable to downstream tools. Power BI can automatically detect and build relationships when a semantic model is created from the warehouse.

Steps to add Table Constraints:

- Open New SQL query on the toolbar.
- Enter the T-SQL code below and press ► Run button to run the script.

```
-- Add primary keys to dimension tables
ALTER TABLE d_Date
    ADD CONSTRAINT PK_d_Date PRIMARY KEY NONCLUSTERED (DateKey) NOT ENFORCED;

ALTER TABLE d_Store
    ADD CONSTRAINT PK_d_Store PRIMARY KEY NONCLUSTERED (StoreKey) NOT ENFORCED;

ALTER TABLE d_Product
    ADD CONSTRAINT PK_d_Product PRIMARY KEY NONCLUSTERED (ProductKey) NOT ENFORCED;

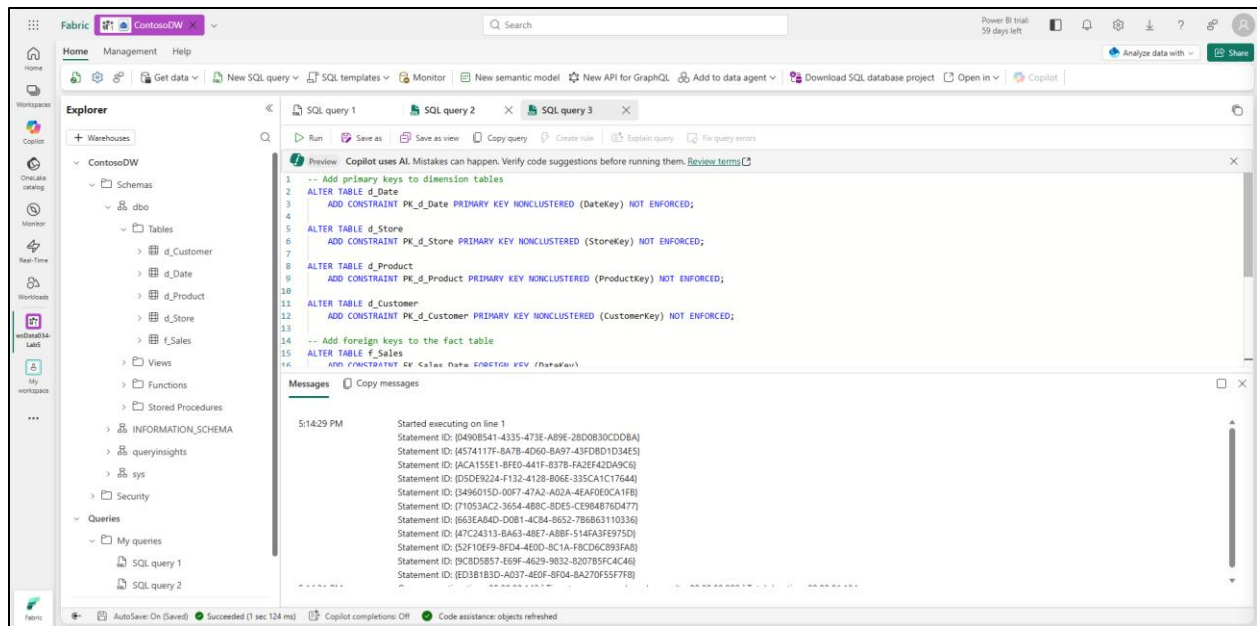
ALTER TABLE d_Customer
    ADD CONSTRAINT PK_d_Customer PRIMARY KEY NONCLUSTERED (CustomerKey) NOT ENFORCED;

-- Add foreign keys to the fact table
ALTER TABLE f_Sales
    ADD CONSTRAINT FK_Sales_Date FOREIGN KEY (DateKey)
        REFERENCES d_Date(DateKey) NOT ENFORCED;

ALTER TABLE f_Sales
    ADD CONSTRAINT FK_Sales_Store FOREIGN KEY (StoreKey)
        REFERENCES d_Store(StoreKey) NOT ENFORCED;

ALTER TABLE f_Sales
    ADD CONSTRAINT FK_Sales_Product FOREIGN KEY (ProductKey)
        REFERENCES d_Product(ProductKey) NOT ENFORCED;

ALTER TABLE f_Sales
    ADD CONSTRAINT FK_Sales_Customer FOREIGN KEY (CustomerKey)
        REFERENCES d_Customer(CustomerKey) NOT ENFORCED;
```



6. Load Sample Data

Now that your tables have constraints in place, the next step is to load sample data so you can query the star schema and see how it behaves.

- Open New SQL query on the toolbar.
- Enter the T-SQL code below and press ► Run button to run the script.
- The script inserts sample records into every dimension table (Date, Store, Product, Customer) and into the fact table (Sales).
- Once loaded, you can start querying the star schema to test relationships and run analytics.

```

-- Load date dimension data
INSERT INTO d_Date VALUES
(20260105, '2026-01-05', 2026, 1, 1, 'January', 5, 'Monday', 2026, 3, 0, 1), (20260112, '2026-01-12',
2026, 1, 1, 'January', 12, 'Monday', 2026, 3, 0, 1),
(20260209, '2026-02-09', 2026, 1, 2, 'February', 9, 'Monday', 2026, 3, 0, 1),
(20260302, '2026-03-02', 2026, 1, 3, 'March', 2, 'Monday', 2026, 3, 0, 1),
(20260406, '2026-04-06', 2026, 2, 4, 'April', 6, 'Monday', 2026, 4, 0, 1),
(20260504, '2026-05-04', 2026, 2, 5, 'May', 4, 'Monday', 2026, 4, 0, 1);

-- Load store dimension data
INSERT INTO d_Store VALUES
(1, 'ST-001', 'Contoso Downtown', 'Flagship', 'Seattle', 'Washington', 'United States', 'West', '2020-
03-15', '2026-01-01', '9999-12-31', 1),
(2, 'ST-002', 'Contoso Mall', 'Standard', 'Portland', 'Oregon', 'United States', 'West', '2021-07-01',
'2026-01-01', '9999-12-31', 1),
(3, 'ST-003', 'Contoso Central', 'Standard', 'Chicago', 'Illinois', 'United States', 'Central', '2019-11-20',
'2026-01-01', '9999-12-31', 1),
(4, 'ST-004', 'Contoso Plaza', 'Express', 'New York', 'New York', 'United States', 'East', '2022-01-10',
'2026-01-01', '9999-12-31', 1);

```

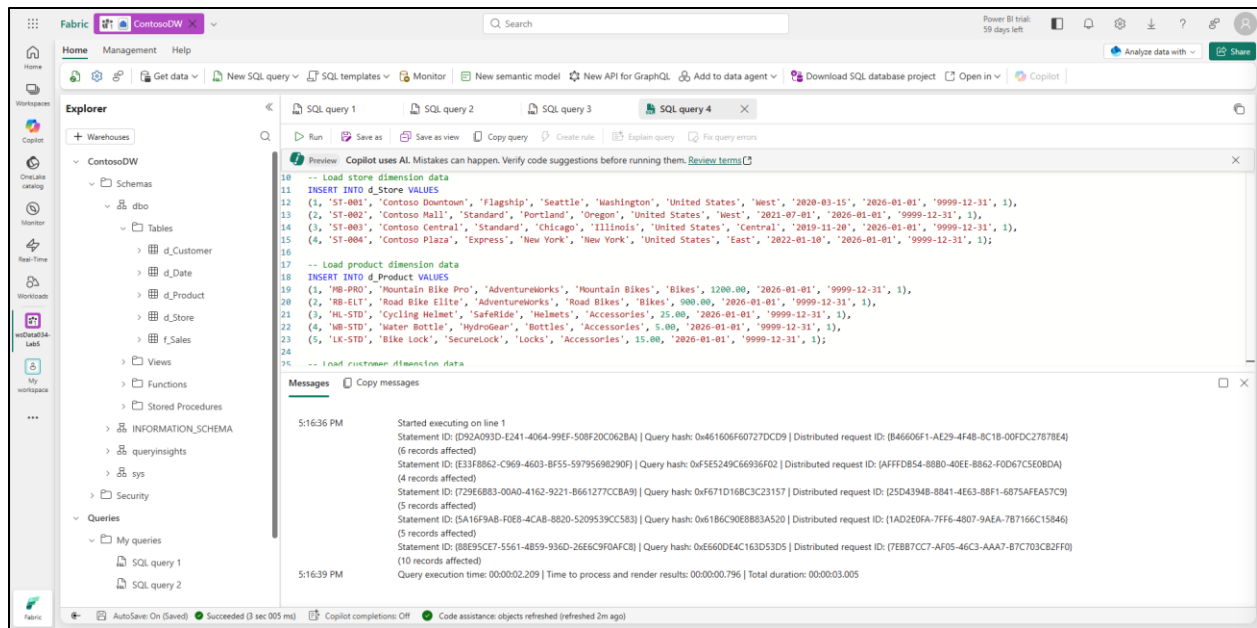
```

-- Load product dimension data
INSERT INTO d_Product VALUES
(1, 'MB-PRO', 'Mountain Bike Pro', 'AdventureWorks', 'Mountain Bikes', 'Bikes', 1200.00, '2026-01-01', '9999-12-31', 1),
(2, 'RB-ELT', 'Road Bike Elite', 'AdventureWorks', 'Road Bikes', 'Bikes', 900.00, '2026-01-01', '9999-12-31', 1),
(3, 'HL-STD', 'Cycling Helmet', 'SafeRide', 'Helmets', 'Accessories', 25.00, '2026-01-01', '9999-12-31', 1),
(4, 'WB-STD', 'Water Bottle', 'HydroGear', 'Bottles', 'Accessories', 5.00, '2026-01-01', '9999-12-31', 1),
(5, 'LK-STD', 'Bike Lock', 'SecureLock', 'Locks', 'Accessories', 15.00, '2026-01-01', '9999-12-31', 1);

-- Load customer dimension data
INSERT INTO d_Customer VALUES
(1, 'Jordan Rivera', 'Premium', 'Seattle', 'Washington', 'United States', 'Gold', '2023-06-15'),
(2, 'Alex Chen', 'Standard', 'Portland', 'Oregon', 'United States', 'Silver', '2024-01-20'),
(3, 'Sam Patel', 'Premium', 'Chicago', 'Illinois', 'United States', 'Gold', '2022-11-05'),
(4, 'Taylor Kim', 'Budget', 'New York', 'New York', 'United States', 'Bronze', '2025-03-12'),
(5, 'Morgan Lee', 'Standard', 'Seattle', 'Washington', 'United States', 'Silver', '2024-08-30');

-- Load fact data (sales transactions)
INSERT INTO f_Sales VALUES
(20260105, 1, 1, 1, 1, 1500.00, 1500.00, 0.00),
(20260105, 1, 3, 1, 2, 35.00, 70.00, 5.00),
(20260112, 2, 2, 2, 1, 1100.00, 1100.00, 100.00),
(20260112, 2, 4, 2, 3, 8.00, 24.00, 0.00),
(20260209, 3, 1, 3, 2, 1500.00, 3000.00, 150.00),
(20260209, 3, 5, 3, 1, 22.00, 22.00, 0.00),
(20260302, 1, 2, 5, 1, 1100.00, 1100.00, 0.00),
(20260302, 4, 3, 4, 4, 35.00, 140.00, 10.00),
(20260406, 2, 1, 2, 1, 1500.00, 1500.00, 75.00),
(20260504, 3, 4, 3, 5, 8.00, 40.00, 0.00);

```



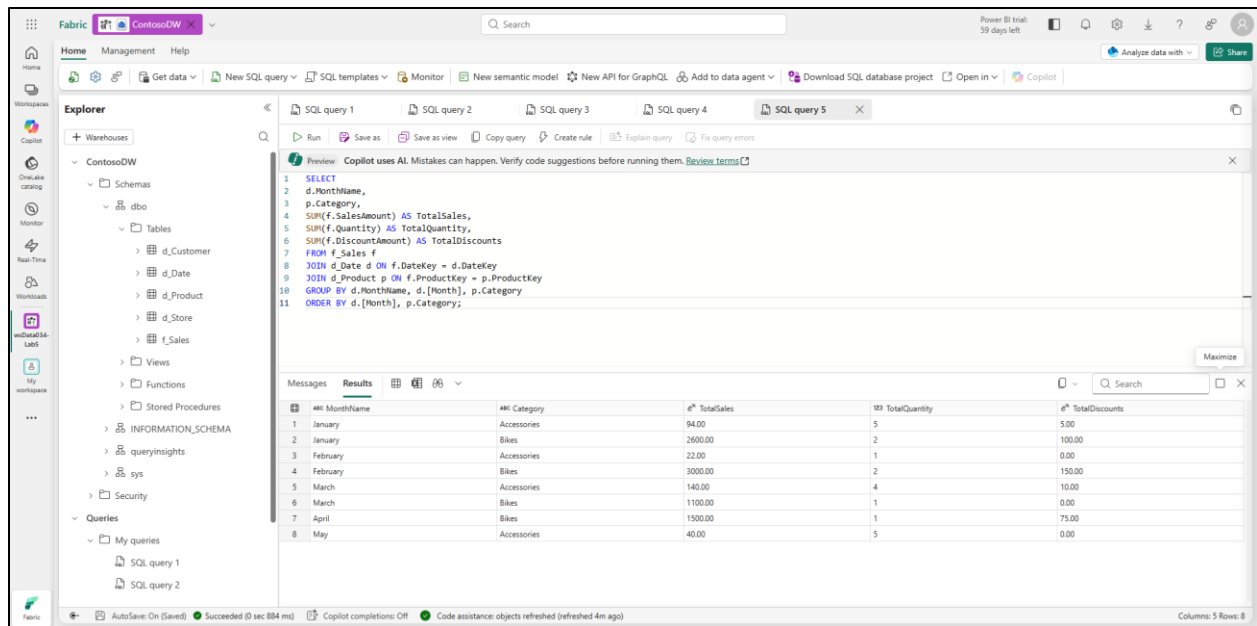
7. Query the Star Schema

The queries below show how to analyze your sales data by joining the fact table to different dimension tables. By pulling descriptive attributes from the dimensions, the fact data becomes easy to summarize and interpret.

Steps to analyze sales by month and product category:

- Open New SQL query on the toolbar.
- Enter the T-SQL code below and press ▶ Run button to run the script.
- The query is joining f_Sales to d_Date and d_Product.
- Use SUM() to total sales, quantity, and discounts
- Group by MonthName and Category to see monthly category performance.

```
SELECT
    d.MonthName,
    p.Category,
    SUM(f.SalesAmount) AS TotalSales,
    SUM(f.Quantity) AS TotalQuantity,
    SUM(f.DiscountAmount) AS TotalDiscounts
FROM f_Sales f
JOIN d_Date d ON f.DateKey = d.DateKey
JOIN d_Product p ON f.ProductKey = p.ProductKey
GROUP BY d.MonthName, d.[Month], p.Category
ORDER BY d.[Month], p.Category;
```



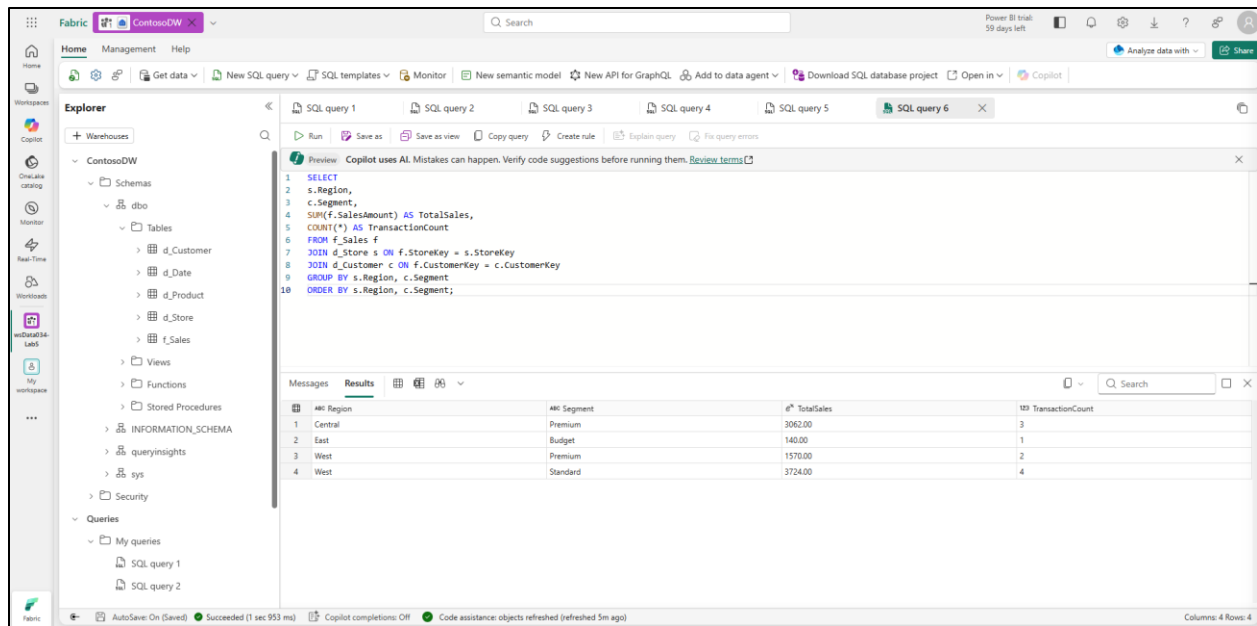
Steps to analyze sales by store region and customer segment:

- Open New SQL query on the toolbar.
- Enter the T-SQL code below and press ► Run button to run the script.
- The query is joining f_Sales to d_Store and d_Customer.
- Aggregate total sales and count transactions.
- Group by Region and Segment to understand geographic and customer trends.
- Review the results. By switching which dimension tables you JOIN and GROUP BY, you can look at the same fact data from different perspectives without modifying the schema.

```

SELECT
    s.Region,
    c.Segment,
    SUM(f.SalesAmount) AS TotalSales,
    COUNT(*) AS TransactionCount
FROM f_Sales f
JOIN d_Store s ON f.StoreKey = s.StoreKey
JOIN d_Customer c ON f.CustomerKey = c.CustomerKey
GROUP BY s.Region, c.Segment
ORDER BY s.Region, c.Segment;

```



8. Implement SCD Patterns

Dimension data doesn't stay the same. Slowly Changing Dimensions (SCD) is a method describing how a dimension table should handle attribute changes over time like a customer's address, a product's price, or a store's region. SCDs ensure dimension updates don't overwrite past values. They preserve every change so your model can answer historical questions accurately.

There are 2 SCD Types:

- Type 1 overwrites the existing value with a new value. This change occurs when there is an error like fixing a customer's name. History is not needed.
- Type 2 adds a new row for each change to keep a full history.

Steps to simulate an SCD Type 2 change:

- Open New SQL query on the toolbar and run the following code below.
- The query expires the current product record and inserts a new version with the updated cost.

```

-- Step 1: Expire the current version of Mountain Bike Pro
UPDATE d_Product
SET ValidTo = '2026-03-01',
IsCurrent = 0
WHERE ProductNaturalKey = 'MB-PRO'
AND IsCurrent = 1;

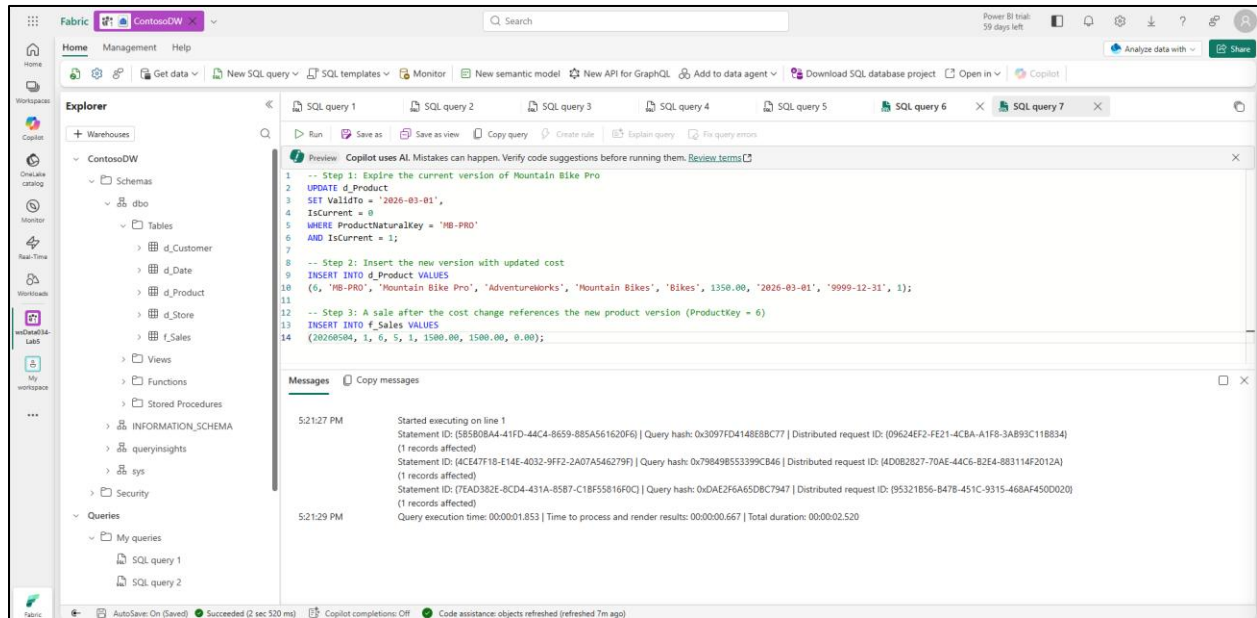
-- Step 2: Insert the new version with updated cost
INSERT INTO d_Product VALUES

```

```
(6, 'MB-PRO', 'Mountain Bike Pro', 'AdventureWorks', 'Mountain Bikes', 'Bikes', 1350.00, '2026-03-01', '9999-12-31', 1);
```

-- Step 3: A sale after the cost change references the new product version (ProductKey = 6)

```
INSERT INTO f_Sales VALUES  
(20260504, 1, 6, 5, 1, 1500.00, 1500.00, 0.00);
```



- Open New SQL query on the toolbar and run the following code below.
- The query demonstrates how SCD Type 2 maintains a complete history of changes

```
SELECT  
d.FullDate,  
p.ProductName,  
p.UnitCost AS ProductCostVersion,  
p.ValidFrom AS CostEffectiveDate,  
f.Quantity,  
f.SalesAmount  
FROM f_Sales f  
JOIN d_Date d ON f.DateKey = d.DateKey  
JOIN d_Product p ON f.ProductKey = p.ProductKey  
WHERE p.ProductNaturalKey = 'MB-PRO'  
ORDER BY d.FullDate;
```

- Review the results. Both records for the original cost and the updated cost are visible. SCD Type 2 keeps historical facts accurate even after changes are made.

The screenshot shows the Microsoft Fabric interface for the ContosoDW workspace. The Explorer pane on the left displays the database schema, including tables like d_Customer, d_Date, d_Product, d_Store, f_Sales, and Views. The main pane shows a SQL query (SQL query 8) that selects data from d_Product and f_Sales, joining them on ProductKey and DateKey. The query results are displayed in a table with 6 columns: FullDate, ProductName, ProductCostVersion, CostEffectiveDate, Quantity, and SalesAmount. The results show four rows of data, including records for 'Mountain Bike Pro' with different cost versions and effective dates.

FullDate	ProductName	ProductCostVersion	CostEffectiveDate	Quantity	SalesAmount
2026-01-05	Mountain Bike Pro	1200.00	2026-01-01	1	1500.00
2026-02-09	Mountain Bike Pro	1200.00	2026-01-01	2	3000.00
2026-04-06	Mountain Bike Pro	1200.00	2026-01-01	1	1500.00
2026-05-04	Mountain Bike Pro	1350.00	2026-03-01	1	1500.00

Steps to simulate an SCD Type 1 change:

- Open New SQL query on the toolbar and run the following code below.
- The query overwrites the product name.

```
UPDATE d_Product
SET ProductName = 'Insulated Water Bottle'
WHERE ProductNaturalKey = 'WB-STD';
```

The screenshot shows the Microsoft Fabric interface for the ContosoDW workspace. The Explorer pane on the left displays the database schema. The main pane shows a new SQL query (SQL query 9) that updates the d_Product table, setting the ProductName to 'Insulated Water Bottle' for the record with ProductNaturalKey = 'WB-STD'. The Messages pane at the bottom shows the execution results, indicating that the query executed successfully, affecting 1 record.

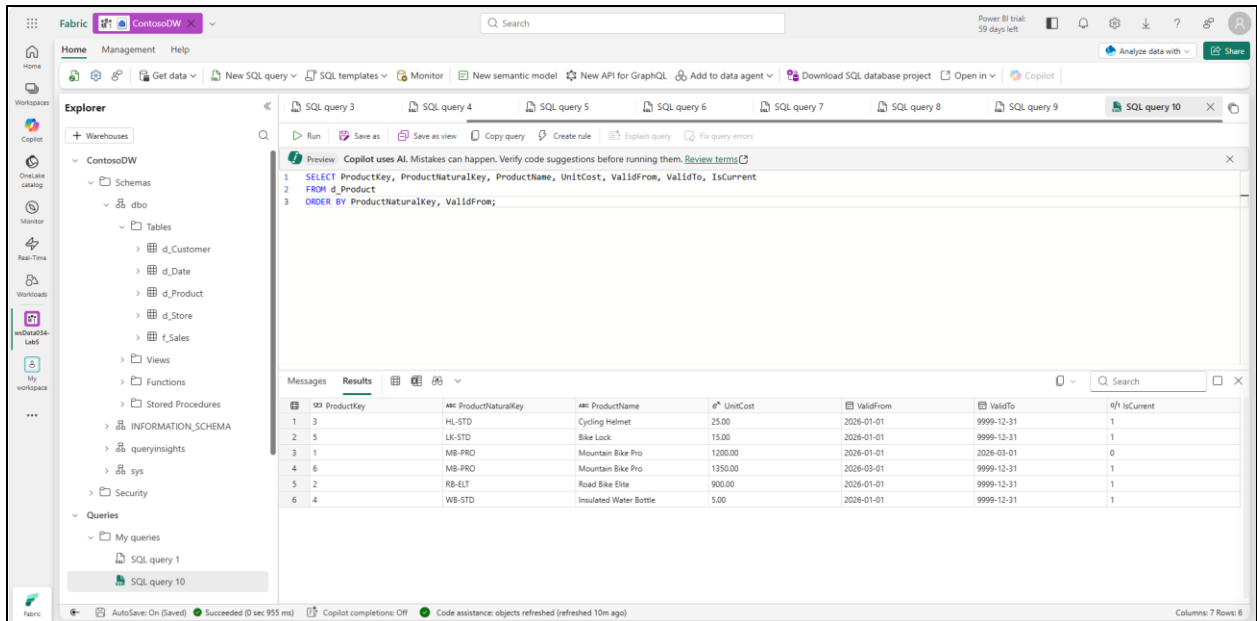
Messages:

```
5:24:03 PM Started executing on line 1
Statement ID: [949A55FB-A306-41C1-8103-ED1E16F8B855] | Query hash: 0xD99472E4BCCC3E8 | Distributed request ID: [5AFF1A4D-D8EE-430A-97B4-66E548EE4241]
(1 records affected)
5:24:04 PM Query execution time: 00:00:00.662 | Time to process and render results: 00:00:00.656 | Total duration: 00:00:01.318
```

- Open New SQL query on the toolbar and run the following code below.
- Confirm that both SCD updates were applied correctly.

```
SELECT ProductKey, ProductNaturalKey, ProductName, UnitCost, ValidFrom, ValidTo, IsCurrent
FROM d_Product
ORDER BY ProductNaturalKey, ValidFrom;
```

- Review the results. MB-PRO is SCD Type 2 with two rows: The original cost (ProductKey1) and the updated cost (ProductKey6). WB-STD is SCD Type 1 with only one row: the original product name is overwritten with the corrected name “Insulated Water Bottle.”



ProductKey	ProductNaturalKey	ProductName	UnitCost	ValidFrom	ValidTo	IsCurrent
1	3	HL-STD	25.00	2026-01-01	9999-12-31	1
2	5	LK-STD	15.00	2026-01-01	9999-12-31	1
3	1	MB-PRO	1200.00	2026-01-01	2026-03-01	0
4	6	MB-PRO	1350.00	2026-03-01	9999-12-31	1
5	2	RB-ELT	900.00	2026-01-01	9999-12-31	1
6	4	WB-STD	5.00	2026-01-01	9999-12-31	1

9. Verify the Design

It is important to confirm that your dimensional model supports the business and can answer key questions needed to make informed decisions.

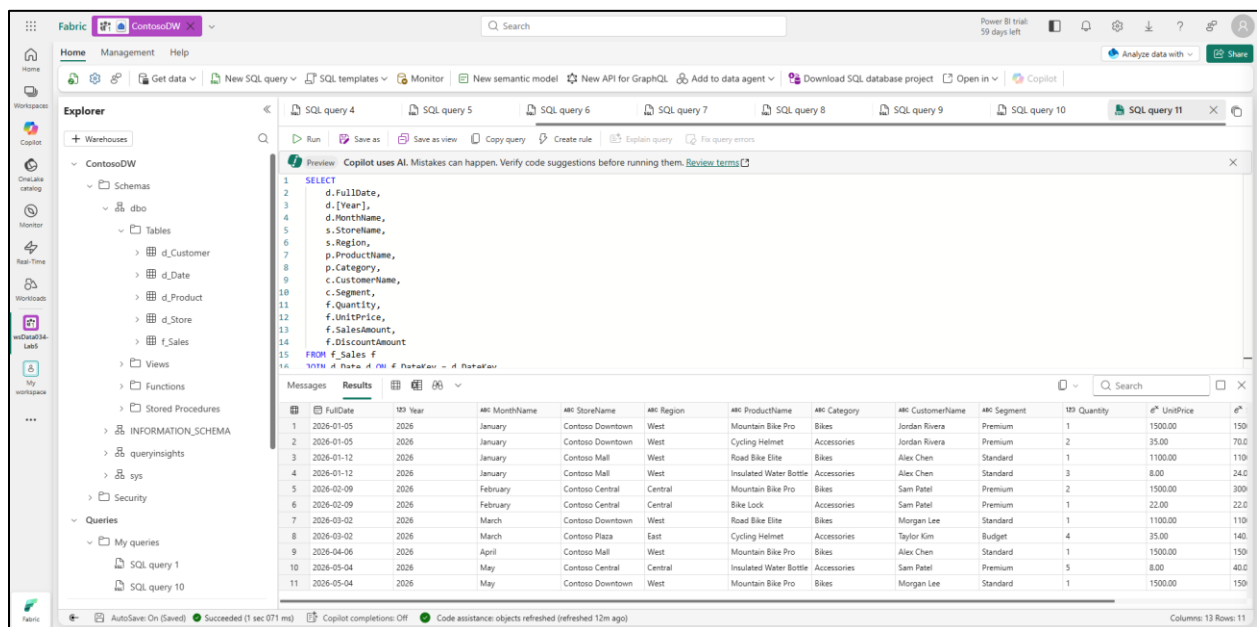
- Open New SQL query on the toolbar and run the following code below.
- By executing this code, it joins the fact table to all four dimensions and allows you to view every part of your star schema.

```
SELECT
    d.FullDate,
    d.[Year],
    d.MonthName,
    s.StoreName,
    s.Region,
    p.ProductName,
```

```

        p.Category,
        c.CustomerName,
        c.Segment,
        f.Quantity,
        f.UnitPrice,
        f.SalesAmount,
        f.DiscountAmount
FROM f_Sales f
JOIN d_Date d ON f.DateKey = d.DateKey
JOIN d_Store s ON f.StoreKey = s.StoreKey
JOIN d_Product p ON f.ProductKey = p.ProductKey
JOIN d_Customer c ON f.CustomerKey = c.CustomerKey
ORDER BY d.FullDate, s.StoreName;

```



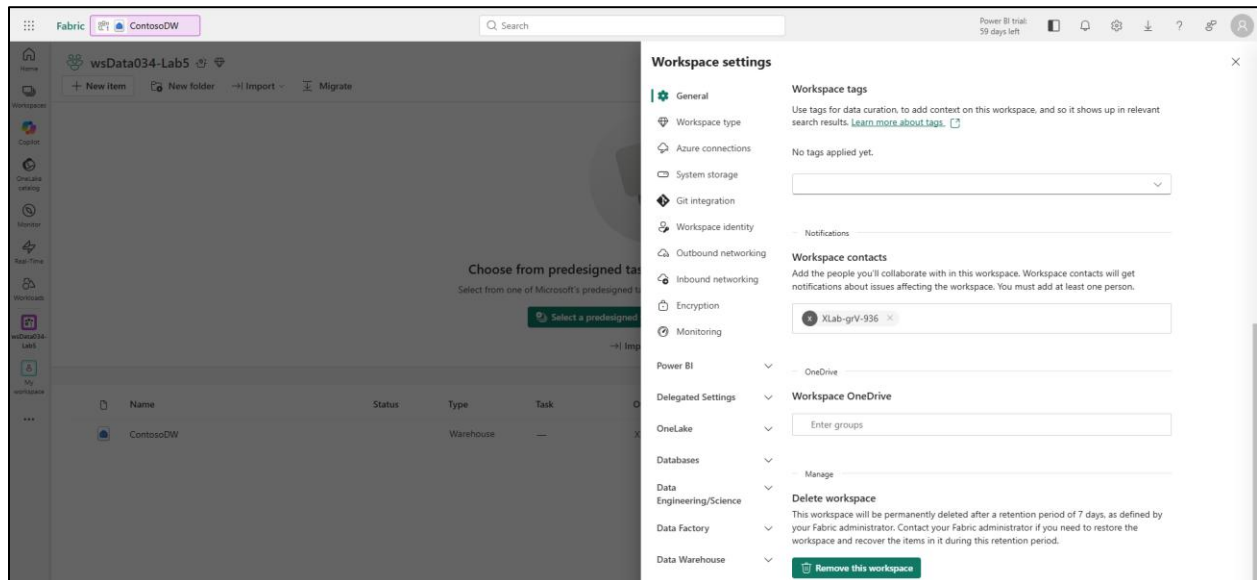
- Review the results. Because your star schema includes four dimension tables, you can analyze the same sales data from four different perspectives. The table below explains this:

Perspective	Dimension Table Used	Description
Sales by time period	Date dimension	Analyze sales across different time period
Sales by location	Store dimension	Provides geographic hierarchy (Region, Country, State, City). Lets you drill down or roll up your analysis.
Sales by product	Product dimension	The product dimension organizes products into a hierarchy (Category, Subcategory, Brand, Product). Lets you understand performance at different levels of detail.
Sales by customer segment	Customer dimension	Attributes like Segment and Loyalty Tier allows you to analyze sales by customer type.

10. Clean up Resources

It is important to delete the resources after completing the lab to stop unnecessary capacity consumption which incur costs.

- Select the icon for your workspace on the left to view all items in the workspace.
- In the toolbar, select Workspace settings.
- In the General section, select Remove this workspace.



11. Reflection

This lab demonstrates how to build a star schema within a data warehouse, load sample data, join a fact table to its dimensions, execute queries within the star schema, and apply Slowly Changing Dimensions (SCD) Type 1 and Type 2 patterns to handle attribute changes.

I learned that data needs to be structured in a way that benefits the business. Dimensional modeling exists because organizations want their data to be usable, reliable and ready for decision-making without the technical complexity. The main idea is that it matches how humans think about business questions. The model is built to be simple, consistent, user-friendly, scalable and reusable in tools like Power BI. The way you construct a dimensional model directly shapes the quality of decisions the business can make because the model determines what questions can be answered.

Dimensional modeling is a design framework to organize data for analytics and reporting. Unlike normalized models used in transactional systems, dimensional models are highly denormalized to deliver fast query performance while keeping it intuitive for users. The star schema offers simplicity with fewer joins which makes it easy to aggregate, slice, filter, and drill down data. This is important to business users because they can analyze the same data from multiple angles. By joining the fact table to different dimensions, users can quickly shift perspectives without modifying the underlying

schema. At the end of the day, what matters most for the business is the outcome. A well-designed dimensional model strengthens the organization's ability to analyze data accurately, uncover meaningful insights, and make confident, data-driven decisions that accelerate performance and push the business forward.